

Battery Voltage Detection

Experiment Objective

This section will guide you through detecting the battery voltage through the ADC interface of the MicroROS-V2 control board. Through this tutorial, you will learn how to use the ESP32-S3's ADC peripheral for single-shot sampling, configure ADC calibration to obtain accurate voltage values, and calculate the battery's actual terminal voltage using the voltage divider ratio.

Experiment Principle

1. Keywords

Keyword	Description
ADC (Analog-to-Digital Converter)	A hardware peripheral that converts analog voltage signals into digital values; the ESP32-S3 defaults to 12-bit resolution with an output range of 0~4095
ADC Attenuation	Configures the ADC input voltage range; 11 dB attenuation corresponds to about 0~2.6 V, protecting the ADC from overvoltage damage
ADC resolution	The number of quantization bits of the ADC; the higher the number of bits, the higher the precision. The ESP32-S3's ADC supports up to 12 bits
Voltage Divider	Reduces a high voltage proportionally through two resistors in series so that the ADC can sample safely; this design has a divider ratio of 4.03:1
ADC Calibration	Uses the calibration parameters in the chip's eFuse to convert raw ADC values into accurate voltage values (mV)
Curve Fitting	The ADC calibration scheme provided by Espressif, which compensates for ADC nonlinearity errors through a curve fitting algorithm to improve measurement precision

2. Technical Architecture

The program samples the battery voltage after voltage division using the ESP32-S3's ADC peripheral. GPIO3 is connected to the battery voltage divider circuit (10KΩ + 3.3KΩ). After the ADC samples the voltage at the divider point, the calibration module converts the raw ADC value into a millivolt voltage, which is then converted into the battery's actual terminal voltage according to the divider ratio (4.03).

Main component files:

- `main/main.c` — main program entry, reads and prints the battery voltage every 500 ms
- `components/battery/battery.c` / `battery.h` — battery voltage detection driver, including ADC initialization, calibration, and voltage calculation

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable	✓ Required (for power supply and serial monitor)
12V lithium battery	✓ Required (connected to the control board power interface)

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Insert the lithium battery into the control board's power interface (XT60 or DC jack)

3. Compile and Flash

```
cd ~/esp/base_samples/voltage
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

After the program starts, it initializes the battery ADC detection module and reads the battery voltage every 500 ms, outputting the voltage value to the serial monitor via logs.

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After correctly flashing and running the program, the serial monitor outputs the following information (the voltage value changes according to the actual battery level):

```
yahboom@VM: ~/esp/base_samples/voltage
yahboom@VM: ~/esp/base_samples/voltage 80x24
I (2773) MAIN: Voltage:12.82V
I (3273) MAIN: Voltage:12.82V
I (3773) MAIN: Voltage:12.82V
I (4273) MAIN: Voltage:12.82V
I (4773) MAIN: Voltage:12.82V
I (5273) MAIN: Voltage:12.82V
I (5773) MAIN: Voltage:12.82V
I (6273) MAIN: Voltage:12.82V
I (6773) MAIN: Voltage:12.82V
I (7273) MAIN: Voltage:12.82V
I (7773) MAIN: Voltage:12.82V
I (8273) MAIN: Voltage:12.82V
I (8773) MAIN: Voltage:12.82V
I (9273) MAIN: Voltage:12.82V
I (9773) MAIN: Voltage:12.82V
I (10273) MAIN: Voltage:12.82V
I (10773) MAIN: Voltage:12.82V
I (11273) MAIN: Voltage:12.82V
I (11773) MAIN: Voltage:12.82V
I (12273) MAIN: Voltage:12.82V
I (12773) MAIN: Voltage:12.82V
I (13273) MAIN: Voltage:12.82V
I (13773) MAIN: Voltage:12.82V
```

- With a 7.4V lithium battery, the normal voltage range is about 7.4V ~ 8.4V
- With an 11.1V lithium battery, the normal voltage range is about 9.6V ~ 12.6V
- If no battery is connected, it may show a value close to 0V or an abnormal value

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/voltage/main/main.c`
- Battery driver: `~/esp/base_samples/voltage/components/battery/battery.h`, `battery.c`

1. Project Structure

```
voltage/
├── CMakeLists.txt           # Top-level CMake project file
├── main/
│   ├── CMakeLists.txt      # main component build configuration
│   └── main.c              # Main program entry
└── components/
    ├── battery/
    │   ├── CMakeLists.txt  # Component configuration (depends on driver, esp_adc)
    │   ├── battery.c       # Battery voltage detection driver implementation
    │   └── battery.h       # Battery voltage detection interface declarations
```

2. Main Program Logic

The execution flow of `app_main` in `main.c`:

1. Print the startup messages "hello yahboom" and "Nice to meet you!"
2. Call `Battery_Init()` to initialize the ADC and start the voltage reading task
3. Enter the main loop, calling `Battery_Get_Voltage()` every 500 ms to get the battery voltage
4. Print the voltage value with `ESP_LOGI` (keeping two decimal places)

3. Key Components

Battery Voltage Detection Driver (battery.h / battery.c)

Pin and ADC configuration:

Parameter definition file: `~/esp/base_samples/voltage/components/battery/battery.h`

```
#define BATTERY_GPIO          3           // Battery voltage detection GPIO
#define ADC_ATTEN_BATTERY     ADC_ATTEN_DB_11 // 11 dB attenuation (range about 0~2.6V)
#define ADC_CHANNEL_BATTERY   ADC_CHANNEL_2 // ADC channel 2 (corresponds to GPIO3)
```

ADC initialization (`Battery_Adc_Init`):

1. Create an ADC unit (`adc_oneshot_new_unit`), configured as `ADC_UNIT_1`
2. Configure the ADC channel parameters:

```
adc_oneshot_chan_cfg_t config = {
    .bitwidth = ADC_BITWIDTH_DEFAULT, // Default bit width (12 bits)
    .atten    = ADC_ATTEN_BATTERY,    // 11 dB attenuation
};
```

3. Configure the ADC channel (`adc_oneshot_config_channel`)
4. Enable ADC calibration (`Battery_Adc_Calibration_Init`), using the curve fitting scheme

ADC calibration initialization (`Battery_Adc_Calibration_Init`):

```
adc_cali_curve_fitting_config_t cali_config = {
    .unit_id = unit,           // ADC_UNIT_1
    .chan = channel,          // ADC_CHANNEL_2
    .atten = atten,           // ADC_ATTEN_DB_11
    .bitwidth = ADC_BITWIDTH_DEFAULT, // Default bit width
};
```

Create the calibration scheme via `adc_cali_create_scheme_curve_fitting`.

Voltage calculation principle:

The control board detects the battery voltage through resistive voltage division; GPIO3 samples the voltage after the division. The divider resistors are 10KΩ and 3.3KΩ, and the divider ratio is:

$$V_{\text{battery}} = V_{\text{gpio}} \times (10\text{K} + 3.3\text{K}) / 3.3\text{K} = V_{\text{gpio}} \times 4.03$$

The corresponding code implementation (`Battery_Update_Voltage`):

```
battery_voltage = gpio_voltage_mv / 1000.0 * 4.03;
```

Voltage reading task (`Battery_Task`):

1. Perform sampling every 100 ms:
 - Call `adc_oneshot_read()` to read the raw ADC value
 - Call `adc_cali_raw_to_voltage()` to convert the raw value into a calibrated voltage (mV)
 - Call `Battery_Update_Voltage()` to calculate the battery terminal voltage based on the divider ratio
2. The task runs on core 1, with a stack size of 3 KB and priority 2

API functions:

Function	Description
<code>void Battery_Init(void)</code>	Initialize battery ADC detection and start the voltage reading task
<code>float Battery_Get_Voltage(void)</code>	Get the current battery voltage, unit: V (volts)

Common Problems

Problem	Cause	Solution
The voltage always displays 0V	No battery connected or ADC initialization failed	Check whether the battery is connected and restart the control board to retry
The voltage value is too low or too high	Wrong divider ratio conversion or calibration failed	Confirm the battery type (7.4V / 11.1V); you can measure the battery terminal voltage with a multimeter to calibrate against it

Problem	Cause	Solution
ADC calibration fails and prints error logs	The chip's eFuse does not store calibration parameters	Some ESP32-S3 chips may lack calibration data; try using <code>ADC_ATTEN_DB_0</code> to reduce the attenuation